[TODO: title]

[TODO: authors]

[TODO: date]

Abstract

[TODO: abstract]

1 Introduction

A posterior over the weights of a transformer is paid for in GPU memory and GPU time. The published record rarely prices it. Variational methods are reported against ensembles trained by other groups, on other cards, through other kernels, and the cost column is missing or is quoted from a framework whose overheads nobody controlled.

We put four uncertainty methods on one network and priced them against each other. Bayes-by-Backprop [1], Vadam [11], a learned low-rank-plus-diagonal Gaussian posterior, and a deep ensemble [14] share a single CUDA implementation of GPT-2 [20]: the same kernels, the same forward and backward pass, the same dataloader handing out the same batches in the same order, the same AdamW hyperparameters, the same precision. Each method's source file contains only how it draws a weight vector and how it updates the parameters behind that draw. Everything else lives in one shared header, so a change to the network reaches all four arms at once and two arms cannot drift apart.

That is the whole design. Any difference between two arms is the posterior, because nothing else is permitted to differ.

Writing the network in CUDA rather than on top of a framework is what makes the cost side legible. A posterior is not an abstraction here; it is a count of parameter-sized buffers and a count of kernel passes over them, and both are visible in the source.

Contributions:

- A controlled comparison of three Bayesian posteriors and a deep ensemble on one network, one corpus, and one card, sharing a single implementation of the network rather than one per arm.
- Cost reported in the units it is paid in: parameter-sized buffers and VRAM, and milliseconds per training step on the same device.
- One shared uncertainty harness. All four arms produce NLPD, ECE, and the aleatoric/epistemic decomposition through the same arithmetic, gated by an exact algebraic identity that fails loudly when the probabilities are wrong.
- An account of the optimisation work each posterior demanded — what had to be derived rather than tuned, what had to be regenerated rather than stored, and which constants are load-bearing.

2 Related work

Variational Bayesian neural networks. Hinton and van Camp [7] priced weight uncertainty as description length and gave the objective its first form. Graves [5] made it run on real networks. Blundell et al. [1] put the reparameterisation of Kingma and Welling [13] inside it, which turned the variational parameters into ordinary backprop targets and produced Bayes-by-Backprop as it is used today. Wen et al. [25] attacked the cost of drawing an independent perturbation per example, which a shared per-step draw does not pay but a large-batch implementation eventually must. The thread establishes the estimator; it does not price it against alternatives running on shared kernels. Our Bayes-by-Backprop arm is Blundell et al.'s method unchanged, and what this paper adds is that its buffer count and its step time sit beside three alternatives measured on the same device.

Optimiser-shaped variational inference. Khan et al. [11] observed that Adam's second moment already estimates a curvature, so it can double as a posterior precision and the posterior costs almost nothing beyond the optimiser state. Osawa et al. [18] scaled natural-gradient variational inference to ImageNet, and Shen et al. [23] carried the same idea to GPT-2-scale training. The thread's claim is that a posterior can be nearly free. We turn it into a measurement: the optimiser-shaped arm runs on the same kernels as an explicit-variance arm and an ensemble, so its step time and its VRAM are directly comparable to theirs.

Post-hoc and covariance-structured posteriors. This is the closest existing work to the low-rank arm. Ritter et al. [21] fit a Kronecker-factored Laplace approximation to the curvature at a trained point, giving a structured covariance without changing the training run. Daxberger et al. [3] turned that into a default recipe — fit the Laplace approximation after the fact, often on the last layer only, and pay nothing during training. Maddox et al. [16] build exactly the covariance this paper's third arm uses, a low-rank factor plus a diagonal, but collect the factor from the SGD trajectory once training is over.

All three are post-hoc by construction. The factor is read off a completed run, so the training loop is untouched and the cost lands at the end. Our low-rank arm inverts that: the rank-r factor A is a variational parameter updated every step by backprop through the sample, so the cost lands in the training loop, where it can be measured per step against mean-field and against an ensemble. The posterior is then fit by the same objective that fits the weights, rather than by a curvature estimate at a point, and the KL term has to be evaluated without ever forming the $P \times P$ covariance — the identities in Section 3.3. What we give up relative to the post-hoc line is its main selling point: those methods can be applied to a model somebody else trained, and this one cannot.

Baselines that win on cost. Deep ensembles [14] need no posterior machinery at all and match or beat variational methods on predictive uncertainty across the benchmarks of Ovadia et al. [19], at E times the parameters and E times the training. That is why an ensemble is the control arm here and not a footnote: it is the thing the other three have to justify themselves against, and it is trained by the same code they are. MC dropout [4] is the other cheap baseline. The network in this paper has no dropout layers, so it is out of scope rather than omitted for a reason.

Evaluation vocabulary. We report calibration as expected calibration error [6] and split predictive uncertainty into aleatoric and epistemic parts in the sense of Kendall and Gal [10]. Ovadia et al. [19] is the reference for how such methods behave under dataset shift and is the standard this paper

does not meet: our off-distribution stream is uniform-random tokens, which tests the direction of the epistemic term and nothing about shifted text (Section 8).

3 Four posteriors over one network

The four arms share one implementation of the network. Kernels, buffer layout, forward pass, backward pass, dataloader, checkpoint format, and the evaluation harness live in a single header; each arm's own source file holds its optimiser and nothing else. A fix to the network lands in all four arms in one edit, and the buffer layout is computed in one place, so an arm cannot quietly disagree with another about where a tensor is.

Held fixed across arms: the architecture and parameter layout, the forward and backward kernels including the cuBLAS calls, the dataloader and the order it emits batches in, the AdamW constants of Section 4, fp32 precision, and the RNG discipline — perturbations are a pure function of (seed, step, index), regenerated in the backward pass instead of stored, and evaluation draws from a stream no training step consumed.

Two things vary: how an arm produces a weight vector for a step, and how it turns the gradient with respect to that vector into a parameter update. The subsections below say only that. Everything a subsection does not mention is the shared network.

3.1 Bayes-by-Backprop

A fully factorised Gaussian posterior over every parameter, with both moments free. Each parameter carries a mean μ_i and an unconstrained ρ_i, and a step draws

$$w_i = \mu_i + \mathrm{softplus}(\rho_i)\,\varepsilon_i, \qquad \varepsilon \sim \mathcal{N}(0, I), \tag{1}$$

one draw shared by the whole batch, reparameterised so that the gradient flows to both moments [13, 1]. The softplus keeps the standard deviation positive and compresses its update: AdamW's step is about the learning rate whatever the gradient's magnitude, so an unconstrained standard deviation initialised near 10^{-3} would move by its own size every step. The prior is a zero-mean Gaussian of standard deviation 0.1, entering through a closed-form KL gradient rather than a sampled estimate.

The objective is the ELBO, $\mathbb{E}_q[\mathrm{NLL}] + \mathrm{KL}(q\|p)/N$. The $1/N$ is not a tuning knob. The KL is a per-dataset quantity while the reported NLL is a per-token mean, so N is the training token count and the weight is read off the loader at startup. Fixing this rather than tuning it mattered: a stale hand-set value carried over from a different corpus cost about a nat of NLL and made the Bayesian arms look categorically worse than they are.

This is the arm with the largest fixed footprint. It needs ten parameter-sized buffers where a deterministic model needs four — μ and ρ, an AdamW moment pair for each, the sampled weights and their gradient, and an accumulator for each moment's gradient — and it runs two AdamW passes per step. Regenerating ε from the index removes an eleventh buffer and its traffic while changing no arithmetic; the one-batch overfit test reaches the same loss either way.

3.2 Vadam

[TODO: Vadam]

3.3 Low-rank plus diagonal covariance

Mean-field assumes every weight is independent of every other. This arm buys back a limited amount of correlation with a covariance of the form

$$\Sigma = AA^\top + \mathrm{diag}(d^2), \qquad A \in \mathbb{R}^{P\times r},\ d \in \mathbb{R}^P,\ r = 8. \tag{2}$$

Σ is $P \times P$ and is never formed. Sampling uses the factors directly,

$$w_i = \mu_i + \sum_{k<r} A_{ik} z_k + d_i \varepsilon_i, \tag{3}$$

with one $z \sim \mathcal{N}(0, I_r)$ drawn per weight matrix per forward pass and shared by all P of its weights — that shared draw is what couples them — and ε drawn per weight from the index-keyed Philox stream.

$d = \mathrm{softplus}(\rho_d)$, for the same reason the mean-field arm uses softplus. A is signed, so softplus cannot bound it, and it needs a separate learning rate instead: 10^{-5} for the variance group against 10^{-4} for the means. One rate cannot serve parameters living at 2×10^{-2} and 10^{-4} at once when the optimiser's step is scale-free.

The prior is $\mathcal{N}(0, I/\lambda)$ with $\lambda = 100$, matching the mean-field arm's prior standard deviation of 0.1. Its KL is evaluated with two identities so that Σ stays factored,

$$\mathrm{tr}(\Sigma) = \|A\|_F^2 + \|d\|^2, \qquad \log\det(\Sigma) = \log\det(M) + \textstyle\sum_i \log d_i^2, \qquad M = I_r + A^\top D^{-1} A, \tag{4}$$

which turns a $P \times P$ determinant into an $r \times r$ one. The gradients follow from the Woodbury identity and reduce to the mean-field gradients at $A = 0$, where the stationary point of the diagonal is $1/\sqrt{\lambda}$, the prior standard deviation.

The posterior covers the four GEMM weight matrices in each block — W_{qkv}, W_{proj}, W_{fc}, W_{fc2}, or 10,616,832 of the model's 11,061,531 parameters. Embeddings, LayerNorm gains and biases, and all biases stay deterministic, and each matrix has its own A, d and z, so there is no correlation across matrices or across blocks.

Sampling is a separate kernel, not a fused GEMM prologue. The network calls cuBLAS, so there is no GEMM of ours to fuse into, and fusing only this arm would make the comparison measure kernel engineering instead of posterior structure. Fusion, if done, belongs in every arm at once.

The rank is a compile-time parameter so the length-r loops unroll into registers, which means a checkpoint has to be evaluated at the rank it was trained at. The checkpoint loader's element-count check catches a mismatch rather than reading the wrong tensor.

3.4 Deep ensemble (control)

$E = 5$ independently trained deterministic models [14], differing only in the seed that initialises their weights. The network has no dropout and the dataloader is sequential, so initialisation is the only source of diversity; there is no bagging, which would shrink each member's effective training set. Architecture, data order, optimiser, learning rate, step count, batch size and precision are identical across members and identical to the other three arms. At inference the members' probabilities are averaged, never their logits.

This arm introduces no kernels at all. Members train through the shared forward and backward pass and the shared AdamW, and its evaluation callback returns a pointer to member s without sampling anything. That is exactly why it is the reference: it is the same network with the Bayesian machinery deleted, so every buffer and every kernel pass the other three add is attributable to their posterior and nothing else.

At evaluation the advantage reverses. A Bayesian arm buys its $(S+1)$-th sample for one forward pass; this arm's S is capped at the number of members it paid to train, and raising it costs another full training run. Evaluating at $S = E$ costs every arm the same forward passes. Getting there does not, and the harness prints the S it used rather than quietly evaluating the arms at different sample counts.

4 Experimental setup

Model. A GPT-2 [20] of the usual pre-LayerNorm transformer shape [24]: width $C = 384$, 6 heads, 6 blocks, vocabulary $V = 27$, maximum context 1024, trained at sequence length $T = 64$ and batch $B = 4$. That is 11,061,531 parameters, of which the six transformer blocks hold about 96%. The small vocabulary is the reason: at GPT-2's BPE vocabulary the token embedding alone would dominate the parameter count and the logit activations, and the experiment would be about an embedding table. Attention uses the IO-aware formulation of Dao et al. [2], so no (B, NH, T, T) score buffer is ever allocated. The deterministic scaffolding — data format, loader, reference implementation — follows llm.c [9] and nanoGPT [8].

Data. text8 [17], character-level over 27 symbols (lowercase `a`–`z` plus space), in the standard split: the first 90M tokens for training, the next 5M for validation, the last 5M for test. The loader walks the corpus sequentially and hands out $B \cdot T = 256$ contiguous tokens per step; there is no shuffling and no bagging, so every arm sees the same tokens in the same order. Each arm trained for 100,000 steps, which is 25.6M tokens, or 28% of the training split.

Hardware and precision. One RTX 2060 Max-Q, compute capability `sm_75`, 6 GB of VRAM, fp32 throughout. The four GEMMs per block go through cuBLAS; everything else is a hand-written kernel. Each kernel was developed and benchmarked on its own in `dev/` and checked against the single-threaded C reference implementation before being brought into the shared header.

Optimiser and RNG. AdamW [12, 15] in every arm, with the same constants: learning rate 10^{-4}, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, weight decay 0, no schedule. Randomness comes from Philox 4x32-10 [22], a counter-based generator: a perturbation is a pure function of (seed, step, weight index). The backward pass regenerates the exact ε the forward pass used from the index alone, so no arm ever stores a perturbation buffer. Evaluation draws come from a seed disjoint from every training stream, so an evaluation sample is never a replay of some training step.

Measuring cost. Step time is wall time around the complete step — host-to-device copy, buffer clears, weight sampling, forward, backward, parameter update — taken with CUDA events on the device that ran it. For the ensemble a step is one member's step, so its training cost is E times the figure. The VRAM number is the sum of the allocations an arm makes, computed from the buffer layout rather than read from a driver tool; it counts what the method requires and excludes allocator slack and context overhead. Section 8 states what the step times are and are not.

Measuring quality. Evaluation loads a checkpoint and never trains. All four arms report through one code path: each supplies only a callback that returns draw s, and the shared harness does the rest, so no two arms can compute the same metric by different arithmetic. Per position, over S draws with predictive distributions p_s and mixture $\bar{p} = \frac{1}{S}\sum_s p_s$, it reports negative log predictive

Arm	Mode	Steps	ms/step	VRAM (GB)	NLL first	NLL last	Gate
BBB	TODO	TODO	TODO	TODO	TODO	TODO	TODO
Vadam	TODO	TODO	TODO	TODO	TODO	TODO	TODO
LowR	TODO	TODO	TODO	TODO	TODO	TODO	TODO
Ens	TODO	TODO	TODO	TODO	TODO	TODO	TODO

Table 1: **[TODO: cost caption]**

density of the mixture and of a single draw, expected calibration error over 15 bins of next-token argmax confidence [6], and the decomposition [10]

$$\underbrace{H[\bar{p}]}_{\text{total}} = \underbrace{\tfrac{1}{S}\textstyle\sum_s H[p_s]}_{\text{aleatoric}} + \underbrace{\tfrac{1}{S}\textstyle\sum_s \mathrm{KL}(p_s \,\|\, \bar{p})}_{\text{epistemic}}, \tag{5}$$

in nats, accumulated in double precision. Samples are mixed as probabilities, never as logits: a logit average is a geometric mean and carries the wrong uncertainty.

Equation 5 is an identity for any $S \geq 1$ and any way of drawing the samples, and both halves of the right-hand side are computed directly rather than one being derived from the other. Asserting it therefore tests the implementation and not the model: because the algebra does not depend on the posterior, a violation means the probabilities are wrong — an unnormalised softmax row, a stale buffer, a sample callback that never wrote — and can never mean the posterior is a poor fit. The harness asserts it at 10^{-5} for every arm and exits nonzero.

Samples, splits, and the off-distribution stream. S is a parameter for the Bayesian arms, where an extra draw costs one forward pass. It is not a parameter for the ensemble, whose s-th sample is its s-th trained member; that arm evaluates at $S = E$ and says so. Every reported number carries its split and its S; neither is ever inferred. The off-distribution stream is uniform-random token ids from a fixed seed, evaluated over the same number of positions as the in-distribution stream. It is a direction, not a second domain: epistemic uncertainty has to rise on it, and the harness warns when it does not.

5 What the posterior costs

[TODO: cost results]

6 What the posterior buys

[TODO: quality results]

7 Discussion

[TODO: discussion]

Arm	Split	S	NLPD (single draw)	NLPD (mixture)	ECE
BBB	TODO	TODO	TODO	TODO	TODO
Vadam	TODO	TODO	TODO	TODO	TODO
LowR	TODO	TODO	TODO	TODO	TODO
Ens	TODO	TODO	TODO	TODO	TODO

Table 2: **[TODO: quality caption]**

Arm	Split	S	Total	Aleatoric	Epistemic (ID)	Epistemic (OOD)
BBB	TODO	TODO	TODO	TODO	TODO	TODO
Vadam	TODO	TODO	TODO	TODO	TODO	TODO
LowR	TODO	TODO	TODO	TODO	TODO	TODO
Ens	TODO	TODO	TODO	TODO	TODO	TODO

Table 3: **[TODO: uncertainty caption]**

8 Limitations

Scale is a choice, and it is small. 11,061,531 parameters on a 6 GB card. The scale was picked so that all four arms fit at once: the low-rank arm and the five-member ensemble are the memory-hungry ones, and a model large enough to fill the card would have priced them out and destroyed the comparison. Nothing here should be read as a claim about billion-parameter models. What transfers is the ratio between arms on one implementation, not the absolute cost. The configuration carries two smaller caveats. The character-level vocabulary of 27 keeps the token embedding negligible, which is deliberate, but it means no result here says anything about a posterior over a large embedding table. And the position embedding is sized for a 1024-token context while training runs at 64, so about 3% of the parameters never see a data gradient and are held at the prior by the KL term alone, while still costing posterior state in three of the four arms.

Nothing is converged. 100,000 steps at 256 tokens per step is 25.6M tokens against a 90M token training split — 28% of one epoch. Every number is what a method does at a fixed step budget, not what it converges to, and the ranking between arms is free to change with more steps. The budget is equal across arms, which is what the comparison needs; it is not sufficient for any arm.

Step times are lower bounds, not controlled measurements. The runs span two sessions on a card with a cooling fault. It throttled to 46–51 W at 87–89 °C against a board budget of 65–80 W, and the low-rank arm's step time drifted from 52 to 65 ms within a single run with no change in the work being done. A reported figure is therefore a lower bound on what that arm actually cost per step during its own run, and the two sessions did not throttle alike, so the arms were not clocked under matched conditions. Read the step times as orders of magnitude between arms. A gap of a few milliseconds is not a result. What a healthy card of this class would deliver is not measured here at all.

$n = 1$, and the noise floor is unknown. One training run per arm. No error bars, no seed sweep. Worse, the runs are not reproducible even in principle: three backward kernels — `encoder_backward`, `layernorm_backward` and `attention_backward` — accumulate gradients with `atomicAdd`, so the

order of floating-point additions varies between runs. Two identical invocations diverge. Nothing in this paper measures how large that divergence grows over 100,000 steps, so we cannot say which differences between arms exceed it. Small gaps should be treated as unresolved.

The off-distribution stream is synthetic. Uniform-random token ids, not shifted text. It is a direction check: sample disagreement has to rise where the input carries no structure, and an arm that fails it is not capturing model uncertainty at all. It is not a distribution shift benchmark, and it supports no comparison with the shift literature [19]. A second text domain would be the minimum needed for that.

Noise scale and posterior quality are confounded. Loss currently tracks the injected noise scale monotonically across arms, so an arm that samples less aggressively looks better on predictive metrics for a reason that has nothing to do with the quality of its posterior. As reported, the experiment cannot separate the two. Separating them needs a sweep over the noise scale within each arm and a comparison of curves rather than of single points; that runs against saved checkpoints and needs no retraining.

Not every code path carries a correctness argument. Each kernel was validated against the CPU reference implementation, but that reference is a single-block model, so the multi-block loop at $L = 6$ has no independent check. Its only evidence is behavioural: the one-batch overfit test still drives the loss to 0.0070 at $L = 6$. A finite-difference gradient check on a tiny multi-block configuration would close the gap and has not been run.

The arms are not identical in what they cover. The low-rank arm places its posterior over the four GEMM weight matrices per block and leaves embeddings, LayerNorm parameters and biases deterministic; the other three are Bayesian over every parameter. At this configuration that is 96% of the model rather than all of it, so the gap is small, but it is a gap and it flatters the low-rank arm's cost. Relatedly, the ensemble evaluates at $S = E = 5$ by construction while the Bayesian arms evaluate at whatever S is requested; a mixture metric compared across arms at different S compares different sample counts, which is why S is printed with every number.

References

[1] Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight uncertainty in neural network. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *Proceedings of Machine Learning Research*, pages 1613–1622. PMLR, 2015. URL https://proceedings.mlr.press/v37/blundell15.html.

[2] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.

[3] Erik A. Daxberger, Agustinus Kristiadi, Alexander Immer, Runa Eschenhagen, Matthias Bauer, and Philipp Hennig. Laplace redux — effortless Bayesian deep learning. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*, pages 20089–20103, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/a7c9585703d275249f30a088cebba0ad-Abstract.html.

[4] Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48 of *JMLR Workshop and Conference Proceedings*, pages 1050–1059. JMLR.org, 2016. URL http://proceedings.mlr.press/v48/gal16.html.

[5] Alex Graves. Practical variational inference for neural networks. In *Advances in Neural Information Processing Systems 24 (NIPS)*, pages 2348–2356, 2011. URL https://proceedings.neurips.cc/paper/2011/hash/7eb3c8be3d411e8ebfab08eba5f49632-Abstract.html.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 2017. URL https://proceedings.mlr.press/v70/guo17a.html.

[7] Geoffrey E. Hinton and Drew van Camp. Keeping the neural networks simple by minimizing the description length of the weights. In *Proceedings of the Sixth Annual ACM Conference on Computational Learning Theory (COLT)*, pages 5–13. ACM, 1993. doi: 10.1145/168304.168306.

[8] Andrej Karpathy. nanoGPT. https://github.com/karpathy/nanoGPT, 2022.

[9] Andrej Karpathy. llm.c: LLM training in simple, raw C/CUDA. https://github.com/karpathy/llm.c, 2024.

[10] Alex Kendall and Yarin Gal. What uncertainties do we need in Bayesian deep learning for computer vision? In *Advances in Neural Information Processing Systems 30 (NIPS)*, pages 5574–5584, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/2650d6089a6d640c5e85b2b88265dc2b-Abstract.html.

[11] Mohammad Emtiyaz Khan, Didrik Nielsen, Voot Tangkaratt, Wu Lin, Yarin Gal, and Akash Srivastava. Fast and scalable Bayesian deep learning by weight-perturbation in Adam. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 2611–2620. PMLR, 2018. URL https://proceedings.mlr.press/v80/khan18a.html.

[12] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015. URL http://arxiv.org/abs/1412.6980.

[13] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *2nd International Conference on Learning Representations (ICLR)*, 2014. URL http://arxiv.org/abs/1312.6114.

[14] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems 30 (NIPS)*, pages 6402–6413, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/9ef2ed4b7fd2c810847ffa5fa85bce38-Abstract.html.

[15] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *7th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2019. URL https://openreview.net/forum?id=Bkg6RiCqY7.

[16] Wesley J. Maddox, Pavel Izmailov, Timur Garipov, Dmitry P. Vetrov, and Andrew Gordon Wilson. A simple baseline for Bayesian uncertainty in deep learning. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 13132–13143, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/118921efba23fc329e6560b27861f0c2-Abstract.html.

[17] Matt Mahoney. About the test data. http://mattmahoney.net/dc/textdata.html, 2011. Large Text Compression Benchmark; source of `enwik8` and `text8`.

[18] Kazuki Osawa, Siddharth Swaroop, Mohammad Emtiyaz Khan, Anirudh Jain, Runa Eschenhagen, Richard E. Turner, and Rio Yokota. Practical deep learning with Bayesian principles. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 4289–4301, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/b53477c2821c1bf0da5d40e57b870d35-Abstract.html.

[19] Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, D. Sculley, Sebastian Nowozin, Joshua V. Dillon, Balaji Lakshminarayanan, and Jasper Snoek. Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 13969–13980, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/8558cb408c1d76621371888657d2eb1d-Abstract.html.

[20] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[21] Hippolyt Ritter, Aleksandar Botev, and David Barber. A scalable Laplace approximation for neural networks. In *6th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2018. URL https://openreview.net/forum?id=Skdvd2xAZ.

[22] John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. Parallel random numbers: As easy as 1, 2, 3. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pages 16:1–16:12. ACM, 2011. doi: 10.1145/2063384.2063405.

[23] Yuesong Shen, Nico Daheim, Bai Cong, Peter Nickl, Gian Maria Marconi, Clement Bazan, Rio Yokota, Iryna Gurevych, Daniel Cremers, Mohammad Emtiyaz Khan, and Thomas Möllenhoff. Variational learning is effective for large deep networks. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 44665–44686. PMLR, 2024. URL https://proceedings.mlr.press/v235/shen24b.html.

[24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NIPS)*, pages 5998–6008, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[25] Yeming Wen, Paul Vicol, Jimmy Ba, Dustin Tran, and Roger B. Grosse. Flipout: Efficient pseudo-independent weight perturbations on mini-batches. In *6th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2018. URL https://openreview.net/forum?id=rJNpifWAb.